

Reasoning, Not Debate, Repairs Encoding-Fragile Graph Reasoning in LLMs

Itai Paperno

Roi Guri

Shahar Cohen

itaipaperno@mail.tau.ac.il roiguri@mail.tau.ac.il shaharc6@mail.tau.ac.il

Abstract

Large language models answer the same graph question with sharply different accuracy depending only on how the graph is serialized into text - a robustness gap we confirm on Llama-3.3-70B. A natural remedy is multi-agent debate: a Critic checks a Proposer’s reasoning before it is committed. Debate rests on a verification asymmetry - checking a claim should be cheaper than producing one - and graph reasoning is an unusually clean place to test it, because an edge claim is objectively present in the serialization or it is not. Across three tasks, three encodings, and three seeds, we find that debate does raise accuracy and does narrow the encoding gap. However, decomposing the debate into its reasoning scaffold and its verify-and-revise loop shows that the entire gain comes from asking the model to reason: the loop itself contributes nothing. A compute-matched majority vote over independent reasoned answers beats debate on every cell at comparable token cost. Encoding fragility is real and reasoning reduces it - but the multi-agent structure is not why.

1 Introduction

Large language models (LLMs) are increasingly asked to reason over graph-structured data - social networks, knowledge graphs, dependency structures - presented as text. A recurring and uncomfortable finding is that their accuracy is highly sensitive to how the graph is written down: the same graph and the same question, serialized under a different but equally faithful convention, can move accuracy by tens of points (Fatemi et al., 2024). We reproduce this encoding fragility on Llama-3.3-70B: a `connected_nodes` query answered correctly 95.8% of the time under one serialization drops to 48.8% under another, on the same graphs. A strong dependence on notation indicates that the phrasing of the input, not the graph’s structure, is driving the model’s answers.

A natural remedy is multi-agent debate: a Proposer produces an answer and a Critic scrutinizes it before it is committed, a procedure reported to improve factuality and reasoning (Du et al., 2023). Debate rests on an implicit verification asymmetry: recognizing that a claim is wrong is assumed to be cheaper than producing the right claim in the first place (Irving et al., 2018). Yet the evidence is mixed - when debate is placed head-to-head against simple aggregation it does not always win (Choi et al., 2025). These results are hard to interpret because in most reasoning domains the steps a Critic checks are not objectively verifiable.

Graph reasoning removes that obstacle. An atomic claim in a Proposer’s trace - “node 5 is adjacent to node 13” - either appears in the serialized graph or it does not, so a Critic’s job is objectively unambiguous and its behavior can be measured exactly. This makes graphs an ideal setting for the question we ask: does debate reduce encoding fragility, and if it helps at all, is the multi-agent structure the reason?

We run three graph tasks (`edge_existence`, `node_degree`, `connected_nodes`) under three encodings (`adjacency`, `incident`, `friendship`) across three independent seeds, comparing a zero-shot baseline, a Proposer–Critic debate, and a compute-matched majority vote. Our findings are:

- **Encoding fragility is large and significant** on all three tasks, unanimously worst under `friendship` and best under `incident` (§4).
- **Debate raises accuracy and narrows the encoding gap.**
- **The gain is the reasoning, not the debate.** Decomposing debate into its first-turn chain-of-thought and its verify-and-revise loop, the loop contributes nothing: asking the model to reason helps, having a Critic argue about the reasoning does not.
- **A compute-matched majority vote beats debate** on every cell at $1.07\times$ its token cost. The

Critic detects real errors, but acting on its verdicts hurts more than it helps.

The takeaway is a positive result wrapped around a negative one: encoding fragility is real and reasoning genuinely reduces it, but the multi-agent debate structure - the part that is expensive and elaborate - is not what does the work.

2 Related Work

LLM reasoning over serialized graphs. A body of recent work studies how well LLMs answer questions about graphs given as text, and consistently finds that the choice of encoding - how nodes, edges, and structure are rendered into a string - is a primary factor in accuracy. [Fatemi et al. \(2024\)](#) systematically vary graph encodings and show that model performance swings widely across them, with no single encoding dominant across tasks. Building on that work, [Perozzi et al. \(2024\)](#) learn a structured encoding fed directly to the LLM - an alternative to our method, which leaves the text input unchanged and instead has the model reason more at inference time. We take this fragility as our starting point, measuring the accuracy gap across encodings and asking whether reasoning can shrink it.

Multi-agent debate. Debate frames inference as an interaction between models that argue toward a better answer. [Irving et al. \(2018\)](#) introduce debate as a scalable-oversight mechanism: two agents argue over a question and a weaker judge rules between them, the hope being that a limited judge can supervise stronger agents because exposing a flaw is easier than committing one. [Du et al. \(2023\)](#) realize this empirically for LLMs, having several instances propose answers and critique one another over multiple rounds until they converge, which improves factuality, commonsense, and arithmetic reasoning. The mechanism rests entirely on this verification asymmetry, and the assumption is not guaranteed: [Barnes and Christiano \(2020\)](#) identify an obfuscated-arguments problem, in which a debater constructs an argument whose flaw neither side can efficiently locate, so refuting a wrong claim is no cheaper than producing a right one. Our Proposer-Critic setup is a deliberately minimal, two-role instance of this family: a single model produces one answer, then critiques and revises it. We expect graphs to avoid the obfuscated-arguments problem, since every claim is verifiable and any flaw can be caught.

Aggregation as the alternative. A simpler way to spend extra inference compute is to sample several answers independently and aggregate them. Chain-of-thought prompting ([Wei et al., 2022](#)) has the model produce intermediate reasoning steps before its answer, improving accuracy on multi-step problems; this is the same reasoning we isolate as debate’s first turn. Self-consistency ([Wang et al., 2023](#)) samples several such reasoning paths independently and returns the majority answer, reliably improving accuracy - which is exactly our majority-vote arm. This makes voting the natural control for debate: both spend more compute than a single answer, but debate lets the attempts interact while voting keeps them independent. [Choi et al. \(2025\)](#) compare the two and find voting competitive with or better than debate in several settings. We sharpen this comparison in two ways: we match compute between the two arms by measured token cost rather than response count, and we run it where debate should do best - a fragile, checkable task with real errors for the Critic to catch.

3 Setup

Data. We generate undirected Erdős-Rényi graphs with NetworkX ([Hagberg et al., 2008](#)), 5 to 19 nodes each and edge probability drawn per graph, giving a wide spread of densities. For each of three seeds (7, 11, 13) we draw an independent set of 200 graphs, and every graph is rendered under all three encodings, so a per-encoding accuracy difference reflects the encoding alone and all comparisons across encodings are paired by graph.

Tasks and encodings. We study three tasks, spanning boolean, integer, and set answers, and three encodings from [Fatemi et al. \(2024\)](#); Table 1 defines each with an example. The encodings preserve the graph exactly and differ only in surface form, labeling nodes with integers except friendship, which uses popular names. Crossing 3 tasks $\times$ 3 encodings $\times$ 3 seeds $\times$ 200 graphs gives 9 cells of $n = 600$ each, 5,400 instances per condition.

Model and conditions. All conditions use Llama-3.3-70B-Instruct-Turbo (FP8). We compare three conditions. The baseline is a single zero-shot answer-only response. Debate is a Proposer-Critic loop: the Proposer emits a numbered claim trace and a final answer, the Critic reviews it against the encoding and returns AGREE or REVISE, and the Proposer revises until agreement

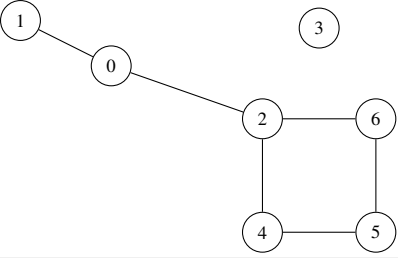
Graph	
	
Encodings	
adjacency	The edges in G are: (0, 1) (0, 2) (2, 4) (2, 6) (4, 5) (5, 6).
incident	Node 0 is connected to nodes 1, 2. Node 2 to 0, 4, 6. Node 4 to 2, 5. Node 5 to 4, 6. Node 6 to 2, 5.
friendship	James and Robert are friends. James and John are friends. John and David are friends. John and Patricia are friends. David and Mary are friends. Mary and Patricia are friends.
Tasks	
edge_existence	Is node 6 connected to node 1? No.
node_degree	What is the degree of node 5? 2.
connected_nodes	List all nodes connected to 2. 0, 4, 6.

Table 1: One graph under the three encodings and the three tasks. All encodings are information-preserving re-serializations of the same graph.

or a turn budget is reached; the Proposer and Critic are the same model in different roles. Its first turn is a single-turn chain-of-thought answer under the same decoding, which lets us separate the reasoning (baseline $\rightarrow$ turn 1) from the verify-and-revise loop (turn 1 $\rightarrow$ final) with no extra runs. Majority vote draws the same Proposer prompt $N=3$ times independently at the model’s shipped sampling defaults and returns the majority answer across the three; it differs from debate in exactly one thing - whether the attempts see each other.

Matched compute. $N=3$ was fixed before the run by matching debate’s measured token spend (debate buys 2.6–2.9 turn-1-equivalents per instance). The realized cost is 1,664 tokens per instance for the vote against 1,554 for debate, so the two arms are compute-matched to within 7% and any advantage the vote shows is not an artifact of the budget. The sampling temperature is Llama’s shipped default, fixed before any result; we report how often the three samples disagreed as a diag-

nostic, but never tuned the temperature to change it.

Metrics and statistics. The primary metric is accuracy; the secondary is encoding fragility, the cross-encoding accuracy spread (max – min) within a task. Because encodings are paired by graph, significance uses paired tests: Cochran’s Q as an overall test across a task’s three encodings, and McNemar’s test for pairwise gaps and for condition-vs-condition accuracy differences; cell accuracies carry Wilson intervals. The Critic diagnostic (does a REVISE track the answer actually being wrong?) is unpaired, one observation per verdict, so it takes a χ^2 association test reported with the signed ϕ . We report the fragility spread descriptively, supported by replication across the three independent seeds rather than by a test of the spread itself. Finally, edge_existence is a single-pair lookup whose trace reduces to one atomic claim, so a claim-by-claim critique has nothing to work on and its baseline sits near ceiling; it behaves as a control rather than a third fragile task, and we report every result both over all 9 cells and over the 6 cells that exclude it.

4 Results and Analysis

Per-cell accuracies, significance, and Critic diagnostics for all nine cells are in [Appendix C](#).

4.1 Encoding fragility is large

Baseline accuracy swings sharply with the serialization on all three tasks (Table 2), reaching a 0.470 gap on connected_nodes between the same graphs. The effect is highly significant on every task under the paired Cochran’s Q test (all $p < 0.001$), and its shape is unanimous: incident is the best encoding and friendship, the one encoding that names nodes rather than numbering them, the worst on all three. This fragility is the phenomenon the rest of the paper asks a reasoning procedure to repair.

4.2 Debate raises accuracy and narrows the gap

Debate raises mean accuracy over the baseline by +0.050 over all nine cells (McNemar $b/c = 351/619$, $p < 0.001$; Table 3), replicating in sign across all three independent seeds. It also narrows the encoding gap (Figure 1): the connected_nodes spread falls $0.470 \rightarrow 0.253$ and node_degree $0.237 \rightarrow 0.107$, reproducing

Task	incident	adjacency	friendship	Spread (max–min)	Cochran Q
connected_nodes	0.958	0.743	0.488	0.470	377
node_degree	0.873	0.655	0.637	0.237	148
edge_existence	0.980	0.938	0.867	0.113	81

Table 2: Baseline accuracy per encoding, with the cross-encoding spread and Cochran’s Q . edge_existence is the near-ceiling control.

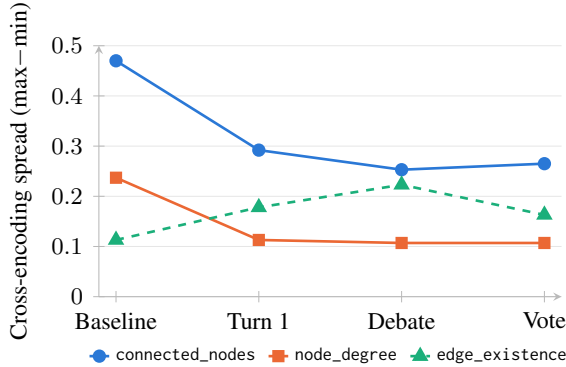


Figure 1: Cross-encoding accuracy spread (max–min) by stage for each task; lower is less fragile.

3/3 on both fragile tasks. The gains concentrate exactly where accuracy was worst - the three largest per-cell gains are the three lowest baseline cells - so debate works by lifting the floor, which is what narrows the spread.

4.3 The gain is the reasoning, not the loop

Debate’s first turn is a single-turn chain-of-thought answer under identical decoding, so the split baseline $\rightarrow$ turn 1 $\rightarrow$ final separates the reasoning from the verify-and-revise loop (Table 3). The reasoning accounts for the entire improvement, worth +0.071 over the baseline, while the loop then reduces accuracy by 0.021. That damage falls on the edge_existence control, where the Critic drags accuracy from 0.922 to 0.803. That task has a single atomic claim: there is nothing to critique, so the Critic only talks a correct answer down. The main takeaway is that asking the model to reason helps; having a Critic argue about the reasoning does not.

Excluding the edge_existence control leaves every conclusion intact (Table 3): without the near-ceiling control, the reasoning gain grows to +0.120, the loop is close to neutral (−0.003), and the compute-matched vote (§4.4) still beats debate (+0.010).

Condition	With control	Without control
Baseline	0.793	0.726
Turn 1 (CoT)	0.864	0.846
Debate (final)	0.843	0.843
Majority vote ($N=3$)	0.875	0.853
Reasoning (base $\rightarrow$ T1)	+0.071	+0.120
Loop (T1 $\rightarrow$ final)	−0.021	−0.003
Vote – debate	+0.032	+0.010

Table 3: Mean accuracy by condition, with and without the edge_existence control. The lower block reports the reasoning, loop, and vote–debate differences.

4.4 A compute-matched vote beats debate

The debate loop requires more compute without adding accuracy, so the natural comparison is against spending that same compute on independent attempts that never interact. Majority vote scores 0.875 against debate’s 0.843 overall (+0.032; $p < 0.001$) at matched compute. The vote is at least as accurate as debate on every task and encoding, and more accurate on three. The full ordering across the arm is baseline 0.793 < debate 0.843 < turn 1 0.864 < vote 0.875, so debate is the worst of the three reasoning conditions and is beaten even by its own single first turn. The vote’s advantage is largest on edge_existence, exactly where we would not expect debate to help. This comparison shows that the interaction between the models is what harms the debate.

4.5 Why the debate fails: precision, not detection

The debate’s failure is not that the Critic cannot tell right from wrong. Split the Proposer’s answers by whether they were correct: revision rescued 13% of the wrong answers but broke 4% of the correct ones. Since correct answers outnumber wrong ones six to one, that 4% (209 answers) is more than the 13% (96), so revising breaks more than it fixes and lowers accuracy overall (Figure 2).

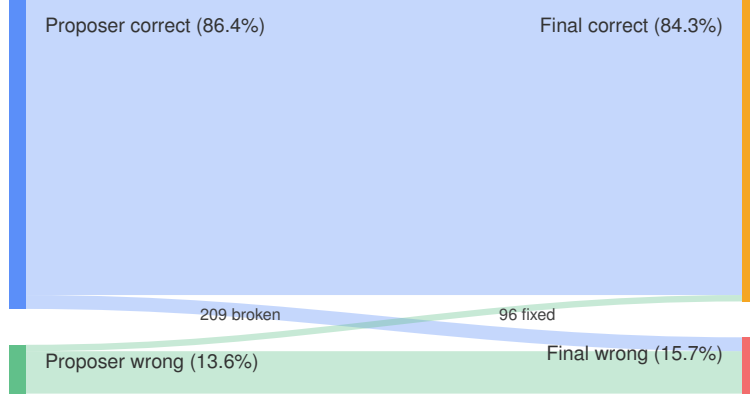


Figure 2: How the Proposer’s first answer maps to the final answer after the debate loop, over 5,400 instances. The crossing bands are the answers the loop changed: correct $\rightarrow$ wrong and wrong $\rightarrow$ correct.

5 Discussion

Reasoning repairs fragility - debate does not.

The encoding gap does shrink under debate, but the decomposition locates the cause: the single-turn reasoning produces most of the encoding variance reduction (Table 3). What reduces a model’s sensitivity to how a graph is written down is being asked to reason about it, not the debate framework around that reasoning.

The asymmetry holds - the debate still loses.

Debate’s premise - that recognizing a wrong claim is cheaper than producing a right one - was satisfied here: the Critic’s REVISE verdicts land on genuinely wrong answers far more often than on correct ones. Yet the loop still lost accuracy, for a reason that is general. When a model is already mostly correct - as Llama-3.3-70B is here, six right for every one wrong - revision puts more correct answers at risk than it can rescue wrong ones, so it does more harm than good.

Reasoning does the work; neither extra structure earns its compute.

Almost all of the gain over the baseline comes from a single reasoned pass; on top of that the vote adds only a little and the debate loop nothing (Table 3). Between the two ways of spending the extra compute, aggregation wins - majority vote beats debate on every cell, because three independent draws cannot argue each other out of a correct answer, whereas a Critic can - but the vote is only the less wasteful option, not a strong one. This does not refute debate in general: it may still help where the base error rate is high, or where a reliable external verifier replaces the model’s own Critic. On encoding-fragile graph reasoning, at matched cost, reasoning once does

the work and neither extra structure adds much.

6 Limitations and future work

- **Single model and graph regime.** Our results come from one model (Llama-3.3-70B) on small Erdős–Rényi graphs, where the model is already mostly correct. Because the debate hurts precisely when errors are rare, harder tasks and weaker models - where errors are more common - are where the results could be better.
- **Single prompt.** We use one Proposer/Critic wording. The Critic’s precision may depend heavily on it, so a different wording could change how much the debate helps or hurts; testing several would tell us whether our result is a property of debate itself or just of this prompt. We did not pursue this, since the loop fails on precision rather than on how well the Critic detects errors.
- **Debate structure.** We tested one minimal two-role loop. Richer structures - more agents, independent proposers that then interact, or multiple rounds - may improve results.

7 Conclusion

LLM graph reasoning is highly encoding-fragile, and asking the model to reason reduces that fragility. A Proposer–Critic debate appears to help, but the gain is entirely the reasoning it introduces: in most of our testing the verify-and-revise loop was harmful, and a compute-matched majority vote beats it on every cell. The Critic genuinely detects errors, it loses because, against a mostly-correct model, its false alarms cost more than its detections repair. On this task, reasoning helps - and debate is not why.

References

- Beth Barnes and Paul Christiano. 2020. [Debate update: Obfuscated arguments problem](#). AI Alignment Forum.
- Hyeong Kyu Choi, Xiaojin Zhu, and Sharon Li. 2025. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. [Improving factuality and reasoning in language models through multiagent debate](#). *arXiv preprint arXiv:2305.14325*.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. [Talk like a graph: Encoding graphs for large language models](#). In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15.
- Geoffrey Irving, Paul Christiano, and Dario Amodei. 2018. [AI safety via debate](#). *arXiv preprint arXiv:1805.00899*.
- Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan Halcrow. 2024. [Let your graph do the talking: Encoding structured data for LLMs](#). *arXiv preprint arXiv:2402.05862*.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). In *The Eleventh International Conference on Learning Representations (ICLR)*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Appendix A Reproducibility

Code and data. The code, the run configurations, the three frozen datasets, and snapshots of the assembled prompt text are at <https://github.com/roiguri/debate-graph-reasoning>.

Data. Each seed is a committed artifact built by the vendored GraphQA generator (Fatemi et al., 2024): 200 Erdős-Rényi graphs crossed with 3 tasks and 3 encodings, giving 1,800 instances. Runs load these files and never regenerate them. `build_dataset.py -verify` rebuilds each from its recorded spec and checks it against the recorded checksum, so a reader can confirm their copy is the one these results were produced on (seed 7: 7030fa5d, seed 11: f6a648eb, seed 13: c462c051).

Model and decoding. Every condition uses Llama-3.3-70B-Instruct-Turbo, served in FP8 through the Together API. Table 4 gives each arm’s decoding and budget. One frozen prompt wording is used throughout; it is reproduced in full in Appendix B.

Arm	Decoding	Cap	Budget
Baseline	greedy	256	1 response
Debate	greedy	512	10 responses
Vote	$T = 0.6$, $\text{top-}p = 0.9$	512	$N = 3$ draws

Table 4: Decoding and budgets per arm, as set in the run configs. **Cap** is `max_new_tokens`; the vote’s sampling values are the model’s shipped defaults.

What a run records. Every run stores one row per instance, carrying the model’s answer, whether it was correct, and its token counts; debate also stores every turn of its transcript. Alongside them a manifest records the model, the dataset, the decoding, and the token cap that produced those rows. The per-cell tables in this paper are computed from those rows, and the repository ships the code that computes them.

Software. Graph generation uses `networkx` (Hagberg et al., 2008); all three datasets reproduce under Python 3.13.9, `networkx` 3.6.1 and `numpy` 2.5.1. The model is reached over HTTP, so no local inference stack is involved.

Appendix B Debate prompts

Below we describe the prompts used for debate, with `{slots}` marking substituted text.

Every template wraps the same `{question}` block, taken from GraphQA (Fatemi et al., 2024) unchanged: the graph serialized under one of the three encodings, followed by the task’s question and a trailing `A:`. Table 1 shows the three encodings and the three tasks for one graph.

Proposer (turn 1). Sent for turn 1 of the debate and for each majority-vote draw.

Answer the question below using the graph. Build the answer as a numbered list of concise, atomic claims -- each stating exactly one simple, verifiable fact about the graph’s nodes or edges -- then give the final answer. Number your claims 1., 2., 3., and so on, one claim per line. Then write a final line that begins with `ANSWER:` followed by `{answer}`. Write nothing after that line.

`{question}`

The `{answer}` slot, used here and in the Revision template, is the only part of either that depends on the task (Table 5).

Task	<code>{answer}</code>
<code>edge_existence</code>	Yes or No
<code>node_degree</code>	a single integer, the degree
<code>connected_nodes</code>	the connected nodes, comma-separated, written exactly as the graph writes them, or none

Table 5: The `{answer}` slot in the Proposer and Revision templates.

Critic. Sent after each Proposer turn.

Another model is answering the graph question below by writing numbered atomic claims (each about one edge or node) and a final answer. You are the checker; the debate so far is shown.

`{transcript}`

`{review instruction}`

The `{transcript}` slot, used here and in the Revision template, is the question, the Proposer’s turn-1 answer, and any later turns.

Revision. The Proposer’s corrected answer, given the Critic’s verdict.

You are the Proposer in the debate below. A checker verified your latest claims against the graph and found problems. Produce a corrected answer that fixes them, using the whole exchange.

`{transcript}`

Give your corrected answer.

Number your claims 1., 2., 3., and so on, one claim per line. Then write a final line that begins with `ANSWER:` followed by `{answer}`. Write nothing after that line.

Critic review instruction. This slot has two forms. For `node_degree` and `connected_nodes`, where the Proposer’s claims are a set of edges:

Work only from the graph text. Review the Proposer's claims and final answer, in both directions:

- did the Proposer MISS an edge that is in the graph?
- did the Proposer INCLUDE an edge that is not in the graph?

AGREE only if the Proposer's supporting edges are exactly the ones in the graph and the answer follows. Otherwise REVISE and name each wrong edge, quoting it from the graph's edge list. Every problem you raise must cite an edge that actually appears in the list; do not repeat a problem the Proposer has already fixed. Respond in exactly this format and nothing else:

VERDICT: AGREE

or

VERDICT: REVISE

- <an edge the Proposer missed or wrongly included, quoted from the graph's edge list>

For edge_existence, where the question is a single pair lookup:

Work only from the graph text. Look in the graph's edge list for the exact pair the question asks about, in either order. If that pair appears, the answer is Yes; if it does not, the answer is No -- decide this yourself from the list. Then check the Proposer's answer.

AGREE if the Proposer's Yes/No matches the edge list. Otherwise REVISE and ground it: quote the queried edge from the list if it is there, or state that no such edge appears. Every problem you raise must be grounded in the edge list; never invent an edge. Respond in exactly this format and nothing else:

VERDICT: AGREE

or

VERDICT: REVISE

- <the queried edge quoted from the list, or a note that it does not appear>

Appendix C Per-cell results

Table 6 gives each cell’s accuracy under the four conditions. The pooled means in Table 3 and the spreads in Figure 1 are computed from these cells.

Encoding	Baseline	Turn 1	Debate	Vote
connected_nodes				
incident	0.958	0.947	0.940	0.952
adjacency	0.743	0.835	0.817	0.822
friendship	0.488	0.655	0.687	0.687
node_degree				
incident	0.873	0.942	0.935	0.943
adjacency	0.655	0.868	0.850	0.875
friendship	0.637	0.828	0.828	0.837
edge_existence				
incident	0.980	0.978	0.975	0.985
adjacency	0.938	0.922	0.803	0.950
friendship	0.867	0.800	0.752	0.822

Table 6: Per-cell accuracy under each condition. Turn 1 is debate’s first-turn chain-of-thought (also each majority-vote draw’s prompt).

Table 7 reports the Critic as a signal: how often a REVISE lands on a wrong answer, and how often on a correct one. Detection counts whether the Critic flagged a wrong answer, not whether the revision then repaired it: a flagged answer can be revised into another wrong answer, so detection bounds what the loop can rescue rather than measuring it.

Encoding	Detection	False alarm	ϕ
connected_nodes			
incident	0.327	0.102	+0.186
adjacency	0.423	0.201	+0.217
friendship	0.308	0.085	+0.288
node_degree			
incident	0.260	0.070	+0.185
adjacency	0.298	0.102	+0.218
friendship	0.208	0.059	+0.208
edge_existence			
incident	0.857	0.105	+0.406
adjacency	0.957	0.290	+0.537
friendship	0.926	0.161	+0.694
Pooled	0.502	0.132	+0.358

Table 7: Per-cell Critic diagnostic. Detection is $P(\text{REVISE} \mid \text{wrong})$, false alarm is $P(\text{REVISE} \mid \text{correct})$, and ϕ is the association between a REVISE and a wrong answer.